חלק 1: סיווג CIFAR-10 באמצעות רשת CNN

מטרת החלק

מטרת חלק זה הייתה לבנות ולאמן רשת קונבולוציונית לסיווג תמונות ממאגר CIFAR-10. בהמשך למטלה הקודמת, שבה ראינו שרשתות fully connected אינן מנצלות בצורה יעילה את המבנה המרחבי של תמונות, עברנו להשתמש ב־ CNN. ההיגיון בבחירה זו הוא שרשת קונבולוציונית מאפשרת ללמוד תבניות מקומיות באמצעות פילטרים, תוך שמירה על היחסים המרחביים בין פיקסלים סמוכים, במקום לשטח את כל התמונה לווקטור כבר בתחילת העיבוד.

המטרה שלנו לא הייתה רק להגיע לדיוק גבוה, אלא לבצע תהליך הדרגתי ושיטתי של חקירה. תחילה בדקנו כמה ארכיטקטורות CNN בסיסיות, לאחר מכן ביצענו חיפוש ממוקד יותר סביב הארכיטקטורות שהראו ביצועים מבטיחים, ולבסוף בדקנו רכיבים שנועדו לשפר הכללה כמו normalization, augmentation, BatchNorm, learning rate scheduler.

מאגר הנתונים וחלוקה לסטים

מאגר הנתונים CIFAR-10 נטען באמצעות torchvision.datasets.CIFAR10 וכולל תמונות צבעוניות בגודל 3×32×32 השייכות לעשר מחלקות שונות.

סט האימון שימש לעדכון משקלי המודל, סט הוולידציה שימש לבחירת ארכיטקטורות והיפר־פרמטרים, וסט המבחן שימש רק בסוף, לאחר בחירת המודל הסופי. בנוסף, השתמשנו בחלוקה stratified לסט הוולידציה, כך שכל מחלקה תהיה מיוצגת בו בצורה מאוזנת. כאשר השתמשנו באוגמנטציה, היא הופעלה רק על סט האימון, בעוד שסט הוולידציה וסט המבחן עברו רק עיבוד דטרמיניסטי, כמו המרה לטנזור ונרמול. הפרדה זו חשובה כדי למנוע מצב שבו ביצועי הוולידציה או המבחן מושפעים משינויים אקראיים בתמונות.

מימוש רשת ה־CNN

לצורך החקירה מימשנו מחלקה גמישה של CNN כך שניתן היה לשנות את מבנה הרשת בין ניסויים בלי לכתוב מחלקה חדשה בכל פעם. הרשת כללה חלק קונבולוציוני ולאחריו מסווג לינארי. בכל בלוק קונבולוציוני היו שכבת קונבולוציה אחת או יותר, פונקציית ReLU ולאחר מכן MaxPooling להקטנת הממד המרחבי.

לאחר החלק הקונבולוציוני, הפלט עבר Flatten והוזן למסווג שכלל שכבה חבויה ReLU ושכבת פלט בגודל 10 עבור עשר המחלקות. הפרמטרים המרכזיים ששינינו בין הניסויים היו מספר הערוצים בכל בלוק, מספר הקונבולוציות לפני pooling, גודל הקרנל, גודל השכבה החבויה במסווג, קצב הלמידה וגודל ה־batch.

הפרמטרים המרכזיים שנבדקו היו:

conv_channels - מספר מפות הפיצ׳רים בכל בלוק קונבולוציוני. פרמטר זה קובע כמה פילטרים שונים הרשת לומדת בכל שלב.

convs_per_block - מספר שכבות הקונבולוציה שמבוצעות לפני כל פעולת pooling. הגדלת ערך זה מאפשרת עיבוד נוסף של מפות הפיצ׳רים לפני הקטנת הממד המרחבי.

classifier_hidden_size - גודל השכבה החבויה במסווג הסופי.

kernel_size - גודל הפילטר הקונבולוציוני.

Padding - הוספת מסגרת אפסים סביב התמונה או מפת הפיצ׳רים, כדי לשלוט בגודל המרחבי לאחר הקונבולוציה.

conv_stride - גודל הצעד שבו הפילטר זז בזמן הסריקה של התמונה.

learning_rate - קצב הלמידה של האופטימייזר, כלומר גודל העדכון של המשקלים בכל צעד אימון.

batch_size - מספר הדוגמאות שנכנסות למודל בכל mini-batch במהלך האימון.

שלב 1: חיפוש ארכיטקטורה בסיסית

בשלב הראשון בדקנו שבע ארכיטקטורות CNN שונות. כדי לבודד ככל האפשר את השפעת הארכיטקטורה, תנאי האימון נשמרו קבועים עם learning rate של 0.001 batch size של 128 ו־15 epochs לכל מודל.

מודל	רעיון מרכזי	ארכיטקטורה	Best Val Acc	Best Val Loss	Best Epoch
A1	רשת בסיסית עם שני בלוקים	[32, 64], conv/block=1, k=3, classifier=256	0.7120	1.1751	15
A2	הוספת בלוק שלישי	[32, 64, 128], conv/block=1, k=3, classifier=256	0.7562	0.8242	11
A3	רשת רחבה יותר	[64, 128, 256], conv/block=1, k=3, classifier=256	0.7702	0.7419	8
A4	שתי קונבולוציות בכל בלוק	[32, 64, 128], conv/block=2, k=3, classifier=256	0.7802	0.9060	11
A5	הגדלת ה־classifier	[32, 64, 128], conv/block=1, k=3, classifier=512	0.7592	0.8675	10
A6	קרנל גדול יותר	[32, 64, 128], conv/block=1, k=5, classifier=256	0.7302	0.8372	7
A7	שתי קונבולוציות וקרנל 5×5	[32, 64, 128], conv/block=2, k=5, classifier=256	0.7578	1.0773	11

בשלב זה A4 הגיע לדיוק הוולידציה הגבוה ביותר 0.7802 ולכן נראה שהוספת שתי קונבולוציות בכל בלוק תרמה ליכולת הייצוג של הרשת. עם זאת ה־ validation loss של A4 היה 0.9060 גבוה יותר מזה של חלק מהמודלים האחרים. לכן לא התייחסנו אליו כמודל ״מנצח״ באופן מוחלט אלא, כמודל שהציג יכולת סיווג חזקה אך עדיין דרש בחינה נוספת מבחינת יציבות והכללה.

לדוגמה A3, הגיע לדיוק ולידציה מעט נמוך יותר 0.7702 אך עם validation loss נמוך יותר 0.7419. פער זה מדגים את החשיבות של הסתכלות משולבת על accuracy ועל loss: accuracy מודד את שיעור הסיווגים הנכונים, בעוד שה־ loss מושפע גם מרמת הביטחון של המודל בתחזיות שלו. לכן כבר בשלב הראשון לא הסתפקנו רק במדד דיוק, אלא ניסינו להבין גם את אופי הלמידה של כל מודל.

באופן כללי, ראינו שהוספת בלוק שלישי שיפרה את הביצועים בהשוואה בין A1 ל־ A2 כאשר דיוק הוולידציה עלה מ־0.7120 ל־0.7562. בנוסף, הגדלת מספר הערוצים שיפרה את הביצועים בהשוואה בין A2 ל־ A3 כאשר A3 הגיע לדיוק ולידציה של 0.7702. התוצאה של A4 הראתה שגם הגדלת עומק העיבוד בתוך כל בלוק, באמצעות שתי קונבולוציות לפני ה־ pooling יכולה לשפר את הביצועים.

הגדלת המסווג הסופי בלבד, כפי שנעשה ב־ A5 לא הובילה לשיפור משמעותי ביחס למודלים המובילים. A5 הגיע לדיוק ולידציה של 0.7592 כלומר, תוצאה טובה אך נמוכה יותר מ־ A4 ומ־ A3. לכן נראה שבשלב זה השיפור המרכזי הגיע בעיקר מהחלק הקונבולוציוני של הרשת, ולא רק מהגדלת החלק ה־ fully connected בסוף.

לגבי גודל הקרנל, התוצאות לא היו חד־משמעיות. שימוש בקרנל 5×5 בלבד, כפי שנעשה ב־ A6 נתן תוצאה נמוכה יחסית של 0.7302. לעומת זאת, כאשר קרנל 5×5 שולב עם שתי קונבולוציות בכל בלוק, כפי שנעשה ב־ A7 הביצועים השתפרו ל־0.7578. כלומר, קרנל גדול יותר לא שיפר את המודל בפני עצמו, אך בשילוב עם מבנה עמוק יותר בתוך הבלוק הוא עדיין היווה כיוון מעניין לבדיקה נוספת.

לשלב הבא בחרנו ארבע משפחות מודלים A4, A3, A5 ו־ A7. הבחירה לא התבססה רק על הדירוג בטבלה, אלא גם על הרצון להמשיך לבדוק כיוונים שונים: שתי קונבולוציות בכל בלוק, רשת רחבה יותר, הגדלת המסווג הסופי, ושילוב של קרנל גדול יותר עם שתי קונבולוציות בכל בלוק. במובן הזה, שלב 1 שימש כסינון ראשוני של כיוונים מבטיחים ולא כבחירה סופית של המודל.

שלב 2: חיפוש ממוקד לאחר סינון ארכיטקטורות

לאחר שלב 1 ביצענו חיפוש ממוקד יותר סביב ארבע משפחות הארכיטקטורה שנבחרו. בשלב זה לא בדקנו מחדש את כל האפשרויות, אלא בחנו האם שינוי של learning rate ו־ batch size יכול לשפר את המודלים המבטיחים או לשנות את דירוגם.

המשפחות שנבחרו היו:

C1_A4_2conv_per_block_k3 - מבנה המבוסס על A4, הכולל שתי קונבולוציות בכל בלוק עם קרנל 3×3.

C2_A5_larger_classifier_k3 - מבנה המבוסס על A5, שבו הוגדל המסווג הסופי ל־512 יחידות.

C3_A3_wider_network_k3 - מבנה המבוסס על A3, הכולל מספר ערוצים גדול יותר .[64, 128, 256]

C4_A7_2conv_per_block_k5 - מבנה המבוסס על A7, הכולל שתי קונבולוציות בכל בלוק עם קרנל 5×5.

בשלב 1 כבר נבדק השילוב learning rate=0.001 ו־ batch size=128 ולכן בשלב 2 בדקנו שלושה שילובים נוספים: הקטנת ה־ batch ל־64, הקטנת קצב הלמידה ל־0.0005, ושילוב של שניהם. בסך הכול נבדקו 12 קונפיגורציות חדשות, לצד התייחסות לתוצאות ה־ baseline משלב 1.

Model family	Source	Architecture idea	LR	Batch	Stage	Best Val Acc	Val Loss at Best Acc Epoch	Best Epoch	Decision
C1	A4	2 conv/block, k=3	0.001	128	Stage 1 baseline	0.7802	0.9060	11	Best overall baseline
C1	A4	2 conv/block, k=3	0.001	64	Stage 2	0.7756	1.1311	14	Best Stage 2 accuracy
C1	A4	2 conv/block, k=3	0.0005	128	Stage 2	0.7664	1.1187	13	—
C1	A4	2 conv/block, k=3	0.0005	64	Stage 2	0.7728	1.1202	12	Strong candidate
C2	A5	larger classifier	0.001	128	Stage 1 baseline	0.7592	0.8675	10	Baseline
C2	A5	larger classifier	0.001	64	Stage 2	0.7510	1.0546	11	—
C2	A5	larger classifier	0.0005	128	Stage 2	0.7662	0.7999	15	Good loss/accuracy balance
C2	A5	larger classifier	0.0005	64	Stage 2	0.7614	0.9738	14	—
C3	A3	wider network	0.001	128	Stage 1 baseline	0.7702	0.7419	8	Strong baseline
C3	A3	wider network	0.001	64	Stage 2	0.7644	1.1249	10	—
C3	A3	wider network	0.0005	128	Stage 2	0.7668	0.7704	11	Good loss/accuracy balance
C3	A3	wider network	0.0005	64	Stage 2	0.7702	1.0804	14	Strong candidate
C4	A7	2 conv/block, k=5	0.001	128	Stage 1 baseline	0.7578	1.0773	11	Baseline

—	6	0.7784	0.7432	Stage 2	64	0.001	2 conv/block, k=5	A7	C4
—	12	1.1348	0.7608	Stage 2	128	0.0005	2 conv/block, k=5	A7	C4
Lowest Stage 2 loss	6	0.7137	0.7632	Stage 2	64	0.0005	2 conv/block, k=5	A7	C4

מתוך תוצאות שלב 2 התמקדנו בעיקר בשני ממצאים. ראשית, לא נמצא שילוב יחיד של learning rate ו־ batch size שהיה עדיף בכל משפחות המודלים. ב־ C1 המבוסס על A4 עם שתי קונבולוציות בכל בלוק וקרנל 3×3, התקבלו התוצאות החזקות ביותר מבחינת דיוק הוולידציה. מודל ה־ baseline של C1 עם learning rate של 0.001 ו־ batch size של 128 הגיע לדיוק הוולידציה הגבוה ביותר בכלל ההשוואה 0.7802. בנוסף, כאשר הקטנו את ה־ batch size ל־64 עם אותו learning rate התקבל דיוק ולידציה קרוב מאוד 0.7756 אך עם validation loss גבוה יותר 1.1311.

שנית, ראינו שגם מודלים אחרים הציגו יתרונות מסוימים אך, לא עקפו את C1 מבחינתaccuracy . לדוגמה C3 המבוסס על רשת רחבה יותר, הגיע ב־ baseline לדיוק ולידציה של 0.7702 עם validation loss נמוך יחסית 0.7419. בנוסף C4 עם learning rate של 0.0005 ו־ batch size של 64 הגיע ל־ validation loss הנמוך ביותר מבין ניסויי Stage 2 0.7137.אך דיוק הוולידציה שלו היה נמוך יותר 0.7632. לכן היתרון שלו ב־ loss לא פיצה על הירידה בדיוק ביחס ל־C1.

מבחינת פרמטרי האימון, התוצאות מצביעות על כך שהקטנת ה־mini-batch יכולה להשפיע על תהליך הלמידה אך לא באופן אחיד בכל הארכיטקטורות. ב־C1 הקטנת ה־ batch ל־64 שמרה על accuracy גבוה יחסית, אך העלתה את ה־ validation loss. המשמעות היא שהמודל הצליח לסווג נכון מספר רב של דוגמאות, אך ייתכן שהיה פחות יציב או פחות מכויל מבחינת רמת הביטחון בתחזיות. בנוסף learning rate נמוך יותר לא שיפר באופן עקבי את כל המודלים, ולכן לא ניתן להסיק ש־ learning rate נמוך או batch קטן עדיפים באופן כללי. ההשפעה שלהם תלויה בארכיטקטורה ובשילוב ביניהם.

בהתאם לתוצאות החדשות, בחרנו להמשיך עם C1/A4 כבסיס לשלב 3. הבחירה התבססה בעיקר על כך ש־ C1 הציג את דיוק הוולידציה הגבוה ביותר בכלל ההשוואה. עם זאת, הבחירה לא הייתה חד־משמעית לחלוטין, משום שחלק מהמודלים האחרים הציגו validation loss נמוך יותר. לכן פירשנו את C1 כמודל בעל יכולת סיווג חזקה, אך כזה שעדיין עשוי לסבול מ־ overfitting או מחוסר יציבות מסוימת. מסיבה זו, בשלב הבא התמקדנו בשיפור יכולת ההכללה של המודל באמצעות normalization, augmentation, BatchNorm ו־ ,schedulerבמקום להמשיך להגדיל את הארכיטקטורה עצמה.

שלב 3: שיפור ההכללה של המודל הסופי

לאחר שלב 2, השתמשנו במודל C1/A4 כבסיס לשלב השיפור הסופי. מודל זה מבוסס על שתי קונבולוציות בכל בלוק עם קרנל 3×3, והוא נבחר משום שהציג את דיוק הוולידציה הגבוה ביותר בשלבים הקודמים. בשלב זה לא המשכנו לשנות את הארכיטקטורה עצמה, אלא בדקנו האם ניתן לשפר את יכולת ההכללה בעזרת רכיבי אימון ורגולריזציה. לשם כך ביצענו Ablation Study מדורג, שבו כל ניסוי מוסיף רכיב מרכזי אחד ביחס לקודם.

מודל	רכיב שנוסף	Optimizer	BatchNorm	Scheduler	Val Acc	Val Loss at Best Acc Epoch	Best Epoch	החלטה
F0	מודל בסיס C1/A4	Adam	לא	ללא	0.7710	1.1491	13	Baseline
F1	+ normalization	Adam	לא	ללא	0.7846	1.1234	14	שיפור קטן
F2	+ normalization + augmentation	Adam	לא	ללא	0.8324	0.4971	15	שיפור משמעותי
F3	+ normalization + augmentation + BatchNorm	Adam	כן	ללא	0.8466	0.4775	15	שיפור נוסף
F4	+ scheduler	Adam	כן	CosineAnnealingLR	0.8590	**0.4146**	15	תוצאה חזקה מאוד
F5	F4 + AdamW + weight decay	AdamW	כן	CosineAnnealingLR	**0.8602**	0.4163	15	המודל הנבחר

התוצאות מראות ש־normalization בלבד תרם שיפור קטן יחסית ביחס למודל הבסיס: דיוק הוולידציה עלה מ־0.7710 ל־0.7846. עם זאת ה־validation loss נשאר גבוה יחסית ולכן נראה שנרמול הקלט לבדו לא הספיק כדי לשפר באופן משמעותי את ההכללה של המודל.

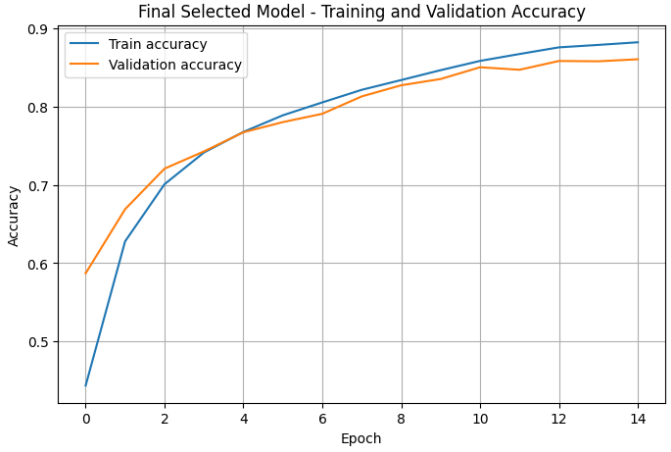
הקפיצה המשמעותית הראשונה התקבלה לאחר הוספת data augmentation דיוק הוולידציה עלה מ־0.7846 ל־0.8324 וה־validation loss ירד בצורה ברורה מ־1.1234 ל־0.4971. מבחינתנו זה היה סימן לכך שחלק משמעותי מהמגבלה של המודל נבע מ־overfitting. ללא אוגמנטציה המודל למד מהר מאוד את דוגמאות האימון, אך התקשה להכליל באותה מידה לסט הוולידציה. האוגמנטציה חשפה את המודל לווריאציות נוספות של אותן תמונות, ולכן עודדה אותו ללמוד תבניות כלליות יותר במקום להסתמך על פרטים ספציפיים מדי.

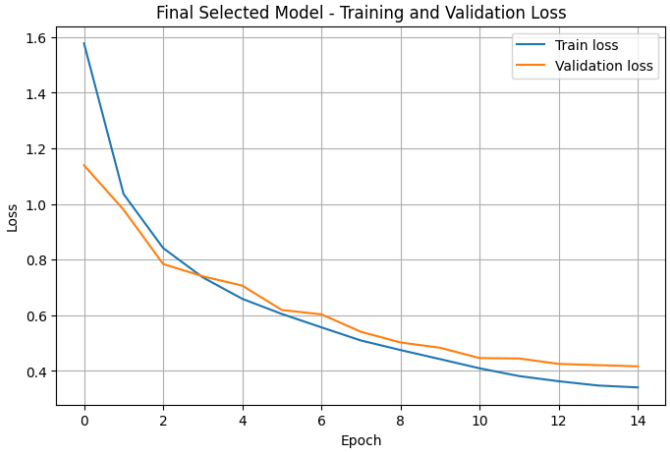
הוספת BatchNorm שיפרה את הביצועים פעם נוספת, והעלתה את דיוק הוולידציה ל־0.8466. שיפור זה מצביע על כך שייצוב האקטיבציות במהלך האימון עזר לתהליך הלמידה והפך אותו ליעיל יותר. לאחר מכן, הוספת CosineAnnealingLR scheduler הובילה לשיפור נוסף ומשמעותי עם validation accuracy של 0.8590

ו־ validation loss של 0.4146. ממצא זה מתאים למוטיבציה של שימוש ב־ scheduler במקום להשתמש בקצב למידה קבוע לאורך כל האימון, קצב הלמידה משתנה בהדרגה, כך שהמודל יכול לבצע עדכונים גדולים יותר בתחילת האימון ועדכונים עדינים יותר לקראת סופו.

בשלב האחרון בדקנו AdamW עם weight decay מודל F5 הגיע לדיוק הוולידציה הגבוה ביותר 0.8602 אך ה־ validation loss שלו היה מעט גבוה יותר מזה של F4 0.4163 לעומת 0.4146. מאחר שהפער בדיוק היה קטן מאוד וגם הפער ב־ loss היה קטן מאוד הבחירה בין F4 ל־ F5 אינה חד־משמעית לחלוטין. עם זאת, במסגרת העבודה בחרנו ב־ F5 כמודל הסופי משום שהוא השיג את דיוק הוולידציה הגבוה ביותר וה־ loss שלו היה כמעט זהה לזה של F4. בנוסף, השימוש ב־AdamW עם weight decay מוסיף רכיב רגולריזציה מפורש, ולכן הוא מתאים למטרה של שלב זה שיפור יכולת ההכללה של המודל.

המודל הסופי F5 כלל את ארכיטקטורת C1/A4 עם normalization, augmentation, BatchNorm, CosineAnnealingLR scheduler, ו־ AdamW optimizer עם weight decay. לאחר בחירתו לפי סט הוולידציה בלבד, הערכנו אותו פעם אחת על סט המבחן הרשמי.





המודל הגיע ל־ test accuracy של 0.8543 ול־ test loss של 0.4376. כלומר, המודל עבר את סף ה־85% על סט המבחן, אך הפער בין דיוק הוולידציה לדיוק המבחן מזכיר שגם לאחר תהליך השיפור, ביצועי המודל על נתונים חדשים הם המדד החשוב ביותר להערכת ההכללה.

חלק 2: Activation Maximization ב-AlexNet

בחלק זה השתמשנו במודל AlexNet מאומן מראש כדי לבחון כיצד רשת קונבולוציונית מייצגת מידע חזותי. בניגוד לחלק הראשון, שבו המטרה הייתה לאמן מודל חדש, כאן משקלי המודל נשארו קבועים והמטרה הייתה לנתח את ההתנהגות של רשת שכבר עברה אימון.

תחילה בדקנו את ביצועי AlexNet על חמש תמונות אמיתיות, פעם אחת עם הנרמול הסטנדרטי של ImageNet ופעם אחת ללא נרמול, כדי להבין את החשיבות של שימוש באותו preprocessing שעליו המודל אומן. לאחר מכן הצגנו את הפילטרים של שכבת הקונבולוציה הראשונה, ובחנו אילו דפוסים ויזואליים בסיסיים הם מזהים. לבסוף השתמשנו ב-Activation Maximization כדי ליצור תמונות קלט שמפעילות בצורה חזקה יחידות, ערוצים, שכבות ומחלקות פלט שונות ברשת. כך ניתן היה להשוות בין ייצוגים בשכבות מוקדמות, שכבות עמוקות ושכבות fully connected.

Image 1: Persian cat



Image 2: Norfolk terrier



Image 3: macaw



Image 4: school bus



Image 5: speedboat



לצורך בדיקת המודל השתמשנו בחמש תמונות צבעוניות מקטגוריות שונות: חתול, כלב, ציפור, רכב/אוטובוס וסירה. התמונות הורדו מתוך ImageNet ונשמרו בתיקיית "images", כך שהן מתאימות למרחב המחלקות שעליו AlexNet אומן. לפני העברת התמונות למודל ביצענו התאמה לגודל הקלט הנדרש: שינוי גודל התמונה, חיתוך למרכז בגודל (224 \ times 224) והמרה לטנזור. לאחר מכן הרצנו את AlexNet בשני תנאים שונים: בתנאי הראשון השתמשנו בנרמול הסטנדרטי שלImageNet , כלומר אותו preprocessing שעליו המודל אומן, בתנאי השני העברנו את התמונות למודל ללא נרמול זה. מטרת ההשוואה הייתה לבדוק עד כמה שלב הנרמול משפיע על התחזיות של מודל מאומן מראש.

תחזיות AlexNet עם וללא נרמול:

תמונה	True label	Prediction with normalization	Prediction without normalization
cat.JPEG	Persian cat	Persian cat	Persian cat
dog.JPEG	Norfolk terrier	Norfolk terrier	West Highland white terrier
bird.JPEG	macaw	macaw	hippopotamus
car.JPEG	school bus	trolleybus	passenger car
ship.JPEG	speedboat	speedboat	broom

Accuracy with normalization: $4/5 = 80\%$

Accuracy without normalization: $1/5 = 20\%$

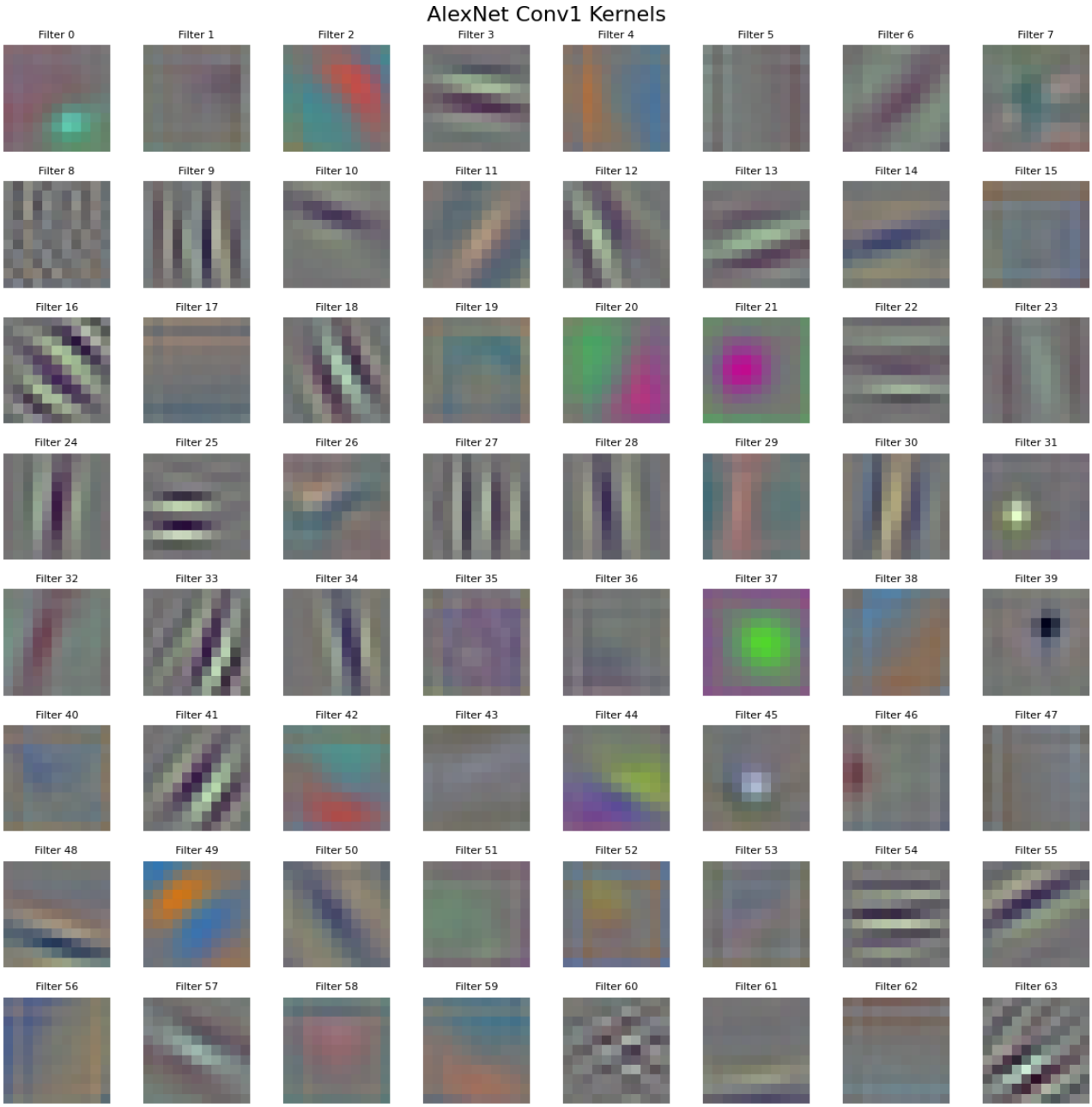
Difference: 60 percentage points

לאחר טעינת AlexNet המאומן מראש, בדקנו את תחזיות המודל על חמש תמונות מ-ImageNet. ההרצה בוצעה בשני תנאים: עם נרמול ImageNet הסטנדרטי וללא נרמול. כאשר השתמשנו בנרמול, המודל סיווג נכון ארבע מתוך חמש התמונות: החתול זוהה כ-Persian cat, הכלב כ-Norfolk terrier, הציפור כ-macaw, והסירה כ-speedboat. התמונה היחידה שלא סווגה בדיוק למחלקת היעד הייתה תמונת הרכב, שבה ה- true label היה school bus אך המודל חזה trolleybus . אף על פי שמדובר בטעות לפי התאמה מדויקת של המחלקות, התחזית עדיין קרובה סמנטית למחלקה הנכונה, משום ששתי המחלקות מתארות כלי תחבורה ציבוריים.

לעומת זאת, כאשר העברנו את אותן תמונות ללא נרמול, ביצועי המודל ירדו באופן משמעותי. רק תמונת החתול סווגה נכון, בעוד ששאר התמונות קיבלו תחזיות שגויות. למשל, תמונת הציפור סווגה כ- hippopotamus ותמונת הסירה סווגה כ-broom. בהתאם לכך, הדיוק עם נרמול היה 80% $= 4/5$, ואילו הדיוק ללא נרמול היה $= 1/5$ 20% . כלומר, השימוש בנרמול שיפר את הדיוק ב-60 נקודות אחוז.

תוצאה זו מדגישה את החשיבות של שימוש באותו preprocessing שעליו המודל אומן. AlexNet אומן על תמונות ImageNet שעברו נרמול לפי ממוצע וסטיית תקן קבועים לכל ערוץ צבע. כאשר מזינים למודל תמונות ללא נרמול, התפלגות הערכים שהמודל מקבל שונה מזו שראה בזמן האימון, ולכן האקטיבציות בשכבות הפנימיות משתנות והתחזיות עלולות להיפגע באופן משמעותי.

ויזואליזציה של פילטרים בשכבת *Conv1* :



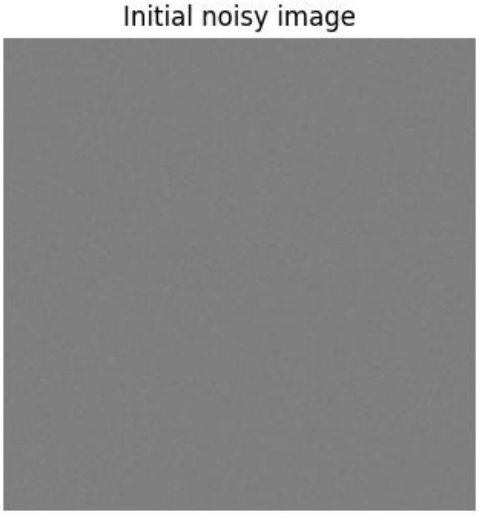
*ויזואליזציה של 64 הפילטרים בשכבת הקונבולוציה הראשונה של AlexNet. כל פילטר הוא בגודל 11 × 11 עם 3 ערוצי צבע, ולכן ניתן להציג אותו כתמונת RGB קטנה לאחר scaling של המשקלים לטווח [0,1].

בשלב הזה הצגנו את המשקלים של שכבת הקונבולוציה הראשונה של AlexNet . שכבה זו מכילה 64 פילטרים, כאשר כל פילטר הוא בגודל 11 × 11 ופועל על שלושת ערוצי הצבע של התמונה. לכן צורת הטנזור של המשקלים היא 64 פילטרים, 3 ערוצי צבע, וגודל מרחבי של 11 × 11, ([64, 3, 11, 11]). מכיוון שהמשקלים אינם ערכי פיקסלים אמיתיים ויכולים לכלול גם ערכים שליליים, ביצענו scaling גלובלי לכל המשקלים כך שהערך המינימלי יוצג כ-0 והערך המקסימלי יוצג כ-1. לאחר מכן הצגנו את כל 64 הפילטרים במבנה של 8 × 8.
מהוויזואליזציה ניתן לראות שהפילטרים בשכבה הראשונה מזהים בעיקר מאפיינים ויזואליים בסיסיים. חלק מהפילטרים נראים כמו קצוות בכיוונים שונים, למשל קווים אופקיים, אנכיים או אלכסוניים. פילטרים אחרים רגישים להבדלים בין צבעים שונים או למעברים בין אזורים בהירים וכהים. תבניות אלה מתאימות לאופי של

שכבה מוקדמת ברשת קונבולוציונית, היא אינה מזהה עדיין אובייקטים שלמים, אלא תכונות בסיסיות כמו צבע, ניגודיות, קצוות ומרקמים פשוטים.
מבחינה רעיונית, יש דמיון בין הפילטרים האלה לבין תופעות שנלמדות במדעי המוח, לדוגמה בשלב עיבוד מוקדם של מערכת הראייה. ב- *V1* קיימים נוירונים הרגישים לאוריינטציה של קווים וקצוות בשדה הראייה. באופן דומה, חלק מהפילטרים בשכבת הקונבולוציה הראשונה של AlexNet נראים כרגישים לכיוונים שונים של קצוות ולמעברים מרחביים. בנוסף, פילטרים מסוימים מציגים רגישות להבדלים בין ערוצי צבע, מה שמזכיר עיבוד מוקדם של מידע צבעוני וניגודיות במערכת הראייה. לכן ניתן לראות שבדומה למערכת הראייה הביולוגית, גם הרשת מתחילה מעיבוד של מאפיינים חזותיים פשוטים, שעליהם נבנים ייצוגים מורכבים יותר בשכבות עמוקות יותר.

ממדי *Conv2* ומגבלות הוויזואליזציה:
לאחר מכן בדקנו גם את ממדי המשקלים של שכבת הקונבולוציה השנייה. צורת טנזור המשקלים שהתקבלה הייתה ([192, 64, 5, 5]) . משמעות הדבר היא שבשכבה השנייה קיימים 192 פילטרים, כאשר כל פילטר מקבל כקלט 64 ערוצים - כלומר את 64 מפות האקטיבציה שהופקו על ידי השכבה הקונבולוציונית הראשונה. הגודל המרחבי של כל *kernel* הוא 5×5.
בניגוד לשכבה הראשונה, לא ניתן להציג את משקלי השכבה השנייה כתמונות RGB פשוטות באותו אופן. בשכבה הראשונה כל פילטר פועל ישירות על שלושת ערוצי הצבע של התמונה, ולכן ניתן להציג אותו כתמונה קטנה בעלת שלושה ערוצים. לעומת זאת, בשכבה השנייה כל פילטר פועל על 64 ערוצי feature maps פנימיים, שאינם מייצגים צבעים ישירות אלא תכונות שנלמדו בשכבה הקודמת. לכן, גם אם נציג פרוסות בודדות מתוך הפילטרים של *Conv2*, הן לא יהיו בעלות משמעות ויזואלית פשוטה כמו פילטרי RGB . מגבלה זו מראה שככל שמתקדמים לשכבות עמוקות יותר, הייצוגים נעשים פחות ישירים מבחינה חזותית ויותר תלויים באקטיבציות שנבנו בשכבות קודמות.

Activation Maximization - יצירת תמונת רעש התחלתית:
בשלב הראשון של Activation Maximization יצרנו תמונת קלט התחלתית מרעש אקראי. ערכי הפיקסלים נדגמו מהתפלגות נורמלית עם ממוצע (0.5) וסטיית תקן (0.01), ולאחר מכן נחתכו לטווח התקין של ערכי תמונה, ([0,1]). בפועל השתמשנו בטנזור בגודל ([224, 224, 3, 1]) כאשר הממד הראשון הוא batch size של תמונה אחת, שלושת הערוצים מייצגים RGB, וגודל התמונה הוא 224×224, בהתאם לקלט הנדרש על ידי AlexNet.
התמונה ההתחלתית נראית כמעט אפורה ואחידה, עם רעש חלש בלבד. בחירה זו מאפשרת להתחיל מנקודת מוצא שאינה מכילה אובייקט ברור, ולבדוק אילו דפוסים ייווצרו כאשר נשנה את הפיקסלים כך שיגדילו את האקטיבציה של יחידה מסוימת ברשת.

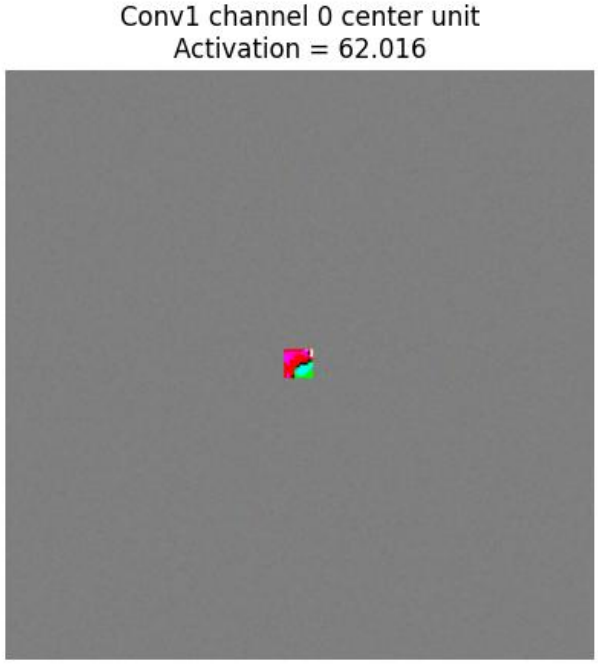


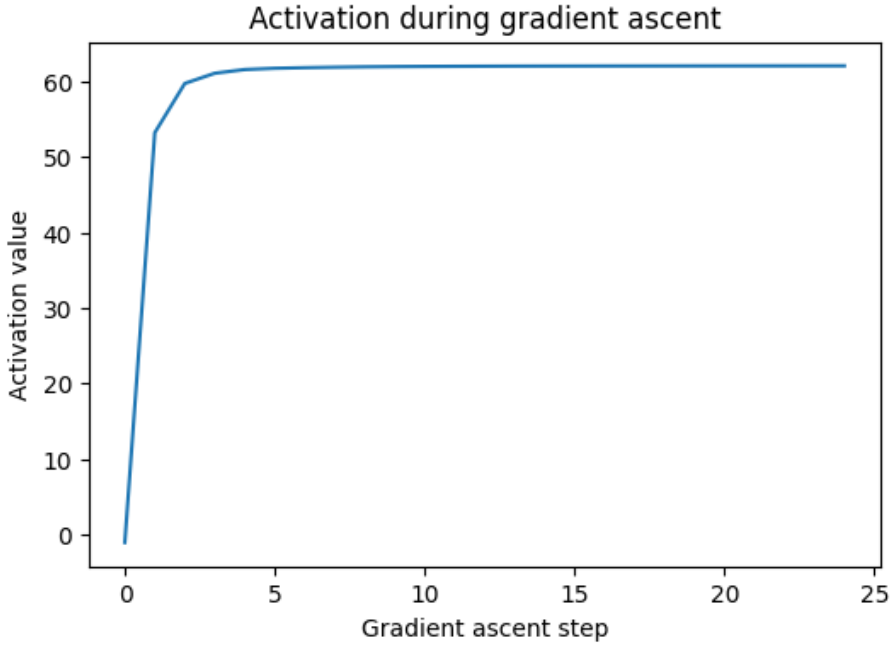
*תמונת הרעש ההתחלתית ששימשה כנקודת מוצא ל- Activation Maximization. ערכי הפיקסלים נדגמו מהתפלגות נורמלית עם ממוצע 0.5 וסטיית תקן 0.01, ולאחר מכן נחתכו לטווח התקין [0,1].

Activation Maximization ליחידה ב-*Conv1*:

לאחר יצירת תמונת הרעש ההתחלתית, ביצענו Activation Maximization עבור יחידה מסוימת בשכבת הקונבולוציה הראשונה של AlexNet . במקום להשתמש בפלט הסיווג הסופי של הרשת, השתמשנו ב- Feature Extractor שמחזיר את האקטיבציות של שכבת *Conv1*. עבור תמונת הקלט התקבלה מפת אקטיבציות בגודל ([1, 64, 55, 55]) כלומר תמונה אחת, 64 ערוצים, וגודל מרחבי של 224 × 224 לכל ערוץ.

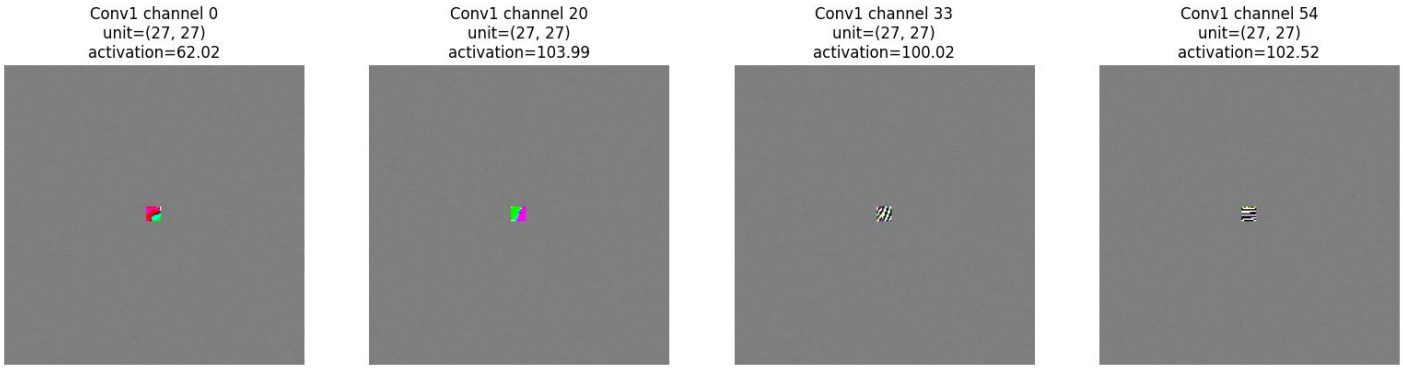
בחרנו את הערוץ הראשון, כלומר (channel=0) , ואת היחידה המרכזית במפת האקטיבציה, כלומר (row=27, col=27) . ערך אקטיבציה יחיד זה שימש כפונקציית המטרה של האופטימיזציה. מכיוון שהמטרה הייתה להגדיל את האקטיבציה של יחידה זו, חישבנו את הגרדיאנט של האקטיבציה ביחס לפיקסלים של תמונת הקלט, ולאחר מכן עדכנו את התמונה באמצעות gradient ascent. כלומר, בכל צעד שינינו את ערכי הפיקסלים בכיוון שמגדיל את האקטיבציה. לאחר כל עדכון חתכנו את ערכי הפיקסלים לטווח התקין ([0,1]), כדי שהתמונה תישאר בתחום ערכים חוקי.





האקטיבציה ההתחלתית של היחידה הייתה $-1.015 \approx$, ולאחר 25 צעדי אופטימיזציה היא עלתה ל- $62.016 \approx$.
בגרף ניתן לראות שהעלייה באקטיבציה הייתה חדה מאוד בצעדים הראשונים, ולאחר מכן הערך התייצב כמעט לחלוטין. כלומר, רוב השינוי המשמעותי בתמונה התרחש בתחילת תהליך ה-gradient ascent.
נציין כי תחילה הרצנו עם 100 צעדים, אך לאחר שראינו כי רוב השינוי קורה בהתחלה צמצמנו ל-25 צעדי אופטימיזציה בלבד.
התמונה שהתקבלה נשארה ברובה דומה לתמונת הרעש האפורה ההתחלתית, אך במרכזה הופיע אזור קטן וצבעוני. תוצאה זו צפויה, משום שמיקסמנו יחידה מרחבית אחת בלבד בשכבת *Conv1*. יחידה בשכבת קונבולוציה מוקדמת מושפעת מאזור מקומי קטן בתמונת הקלט, כלומר מה- receptive field שלה, ולכן רק אזור קטן בתמונה משפיע באופן חזק על האקטיבציה שלה. לכן הכתם המרכזי שנוצר יכול להתפרש כדפוס קלט מקומי שאליו היחידה שנבחרה רגישה במיוחד.
בהשוואה לוויזואליזציה של פילטרים *Conv1*, ניתן לראות קשר רעיוני בין הפילטר לבין תמונת ה- Activation Maximization. הפילטר עצמו מציג את המשקלים שנלמדו, כלומר את הדפוס שאליו הערוץ רגיש. לעומת זאת, תמונת ה- Activation Maximization מציגה קלט שנוצר במיוחד כדי להפעיל יחידה מסוימת באותו ערוץ. לכן שתי הוויזואליזציות אינן אמורות להיות זהות לחלוטין, הפילטר הוא תיאור ישיר של המשקלים, ואילו התמונה הממקסמת היא תוצאה של אופטימיזציה דרך המודל. עם זאת, בשתיהן ניתן לראות דגש על דפוס מקומי וצבעוני, מה שמתאים לכך ששכבת *Conv1* לומדת מאפיינים בסיסיים כמו קצוות, צבעים וניגודיות.
חשוב לציין שלא בהכרח מדובר במקסימום גלובלי יחיד או בדפוס האפשרי היחיד שמפעיל את היחידה. התוצאה תלויה באתחול האקראי של התמונה, בגודל הצעד alpha, במספר צעדי האופטימיזציה, ובחיתוך ערכי הפיקסלים לטווח התקין ([0,1]).

השוואה בין ערוצים שונים ב-*Conv1*:
לאחר שביצענו Activation Maximization עבור היחידה המרכזית בערוץ 0, חזרנו על אותו תהליך עבור ערוצים נוספים בשכבת *Conv1*, כדי לבדוק כיצד ערוצים שונים מגיבים לדפוסים שונים של קלט. בחרנו את הערוצים 0, 20, 33 ו-54, ובכל המקרים מקסמנו את אותה יחידה מרחבית - היחידה המרכזית (27,27). כך ניתן היה להשוות בין הערוצים תוך שמירה על אותו מיקום מרחבי.



*תוצאות Activation Maximization עבור היחידה המרכזית (27,27) בארבעה ערוצים שונים של *Conv1: 0, 20, 33* ו-54. ניתן לראות כי בכל המקרים השינוי ממוקם במרכז התמונה, אך הדפוסים המתקבלים שונים בין הערוצים.

התוצאות מראות כי בכל המקרים התקבל דפוס קטן ומקומי במרכז התמונה, בעוד שרוב התמונה נשאר דומה לרעש האפור ההתחלתי. תוצאה זו צפויה, משום שבכל המקרים מיקסמנו יחידה מרחבית אחת בלבד בשכבת הקונבולוציה הראשונה, ולכן רק ה-receptive field המקומי שלה השתנה באופן משמעותי. עם זאת, הדפוס

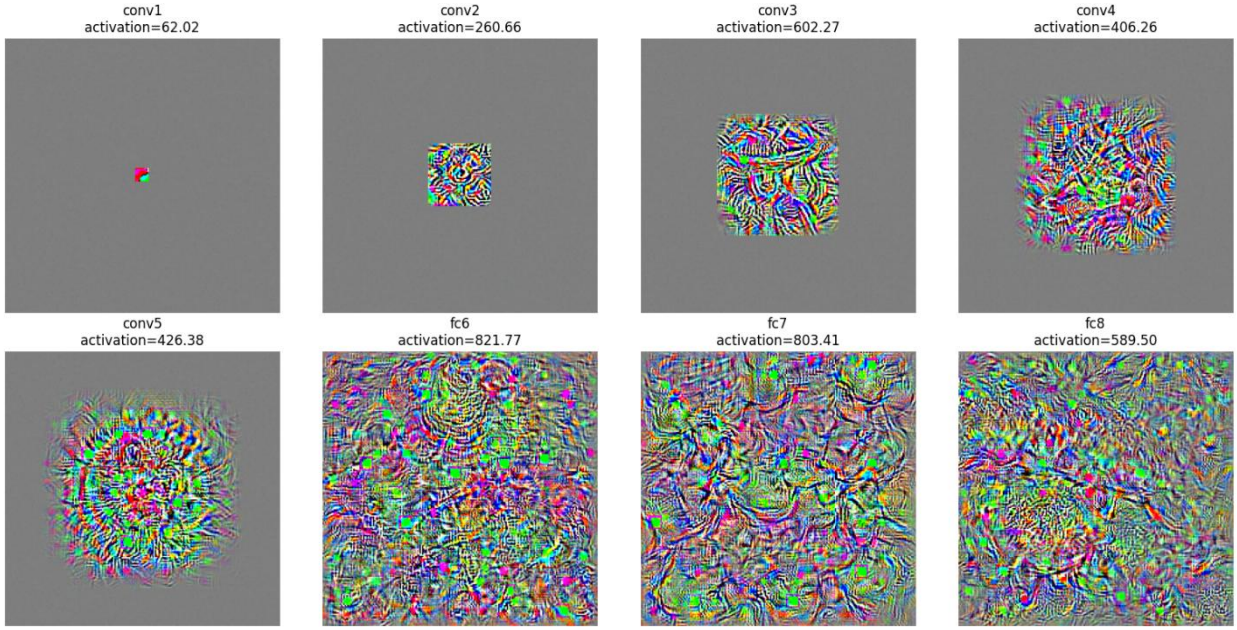
שנוצר במרכז לא היה זהה בין הערוצים השונים. בערוצים מסוימים התקבלו כתמים צבעוניים פשוטים יותר, בעוד שבערוצים אחרים הופיעו דפוסים בעלי מבנה קווי או מפוספס יותר, למשל קווים אלכסוניים או פסים כהים ובהירים.

בנוסף, ערכי האקטיבציה הסופיים לא היו זהים בין הערוצים. עבור channel 0 התקבלה אקטיבציה סופית של כ-62.02, בעוד שעבור channel 20 התקבלו אקטיבציות גבוהות יותר, בסביבות 100-104. גם בגרף 33 ו-54 ניתן לראות שבכל הערוצים האקטיבציה עלתה במהירות בצעדים הראשונים ולאחר מכן הגיעה לרוויה, אך רמת הרוויה הסופית הייתה שונה בין הערוצים.

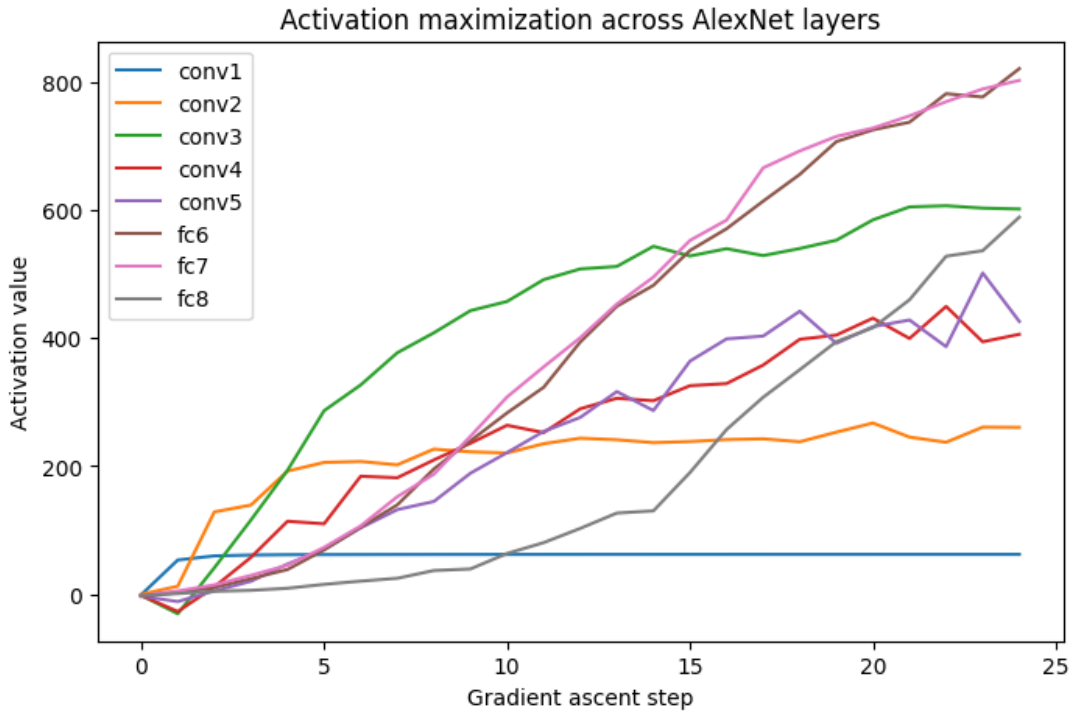
ההבדלים בין התמונות נובעים מכך שכל ערוץ ב-*Conv1* מייצג פילטר אחר, כלומר רגישות שונה לדפוסים ויזואליים שונים. חלק מהפילטרים מגיבים יותר לשילובי צבעים, אחרים לקצוות בכיוון מסוים, ואחרים למרקמים או לפסים. לכן, כאשר ממקסמים ערוצים שונים, מתקבלים דפוסים שונים - כל אחד מהם מייצג את סוג הקלט שמפעיל בצורה חזקה את הפילטר המתאים. מכיוון ששמרנו על אותו מיקום מרחבי, ההבדלים בין התוצאות נובעים בעיקר מזהות הפילטר ולא מהמיקום בתמונה.

השוואה בין שכבות שונות ב-AlexNet:

לאחר שהשווינו בין ערוצים שונים בשכבת *Conv1*, הרחבנו את הניתוח וביצענו Activation Maximization עבור שכבות שונות לאורך AlexNet. עבור כל אחת מחמש שכבות הקונבולוציה (conv1–conv5) מיקסמנו את היחידה המרכזית של הערוץ הראשון, ועבור שלוש השכבות ה- fully connected (fc6–fc8) מיקסמנו את היחידה הראשונה. בכל המקרים התחלנו מתמונת רעש חדשה, כך שהתוצאות עבור השכבות השונות יהיו בלתי תלויות זו בזו.



תוצאות Activation Maximization עבור שכבות שונות ב-*AlexNet: conv1-conv5* ו-*fc6-fc8*. ניתן לראות שככל שמתקדמים לשכבות עמוקות יותר, הדפוסים המתקבלים נעשים מורכבים יותר ומתפשטים על שטח גדול יותר של התמונה.



ערכי האקטיבציה לאורך צעדי ה-gradient ascent עבור שכבות שונות ב-AlexNet.

התוצאות מראות שינוי ברור באופי התמונות המתקבלות לאורך עומק הרשת. עבור *conv1* התקבל דפוס קטן, פשוט ומקומי מאוד במרכז התמונה. ב-*conv2* ו-*conv3* הדפוסים נעשו גדולים ומורכבים יותר, והופיעו בהם מבנים המזכירים מרקמים, פסים, קצוות וצורות גליות. ב-*conv4* ו-*conv5* התבניות נעשו עשירות עוד יותר, ונפרשו על אזור מרחבי גדול יותר. לבסוף, בשכבות *fc6, fc7* ו-*fc8*, כמעט כל התמונה הושפעה, והתקבלו דפוסים מורכבים וצפופים מאוד שהתפשטו על פני רוב שדה התמונה.

ההבדל המרכזי בין התמונות נובע מכך שככל שמתקדמים לשכבות עמוקות יותר, גם ה- receptive field של היחידות גדל וגם רמת המופשטות של הייצוג עולה. בשכבות מוקדמות היחידות רגישות בעיקר למאפיינים חזותיים פשוטים ומקומיים, כגון קצוות, צבעים ומעברים בסיסיים. לעומת זאת, בשכבות עמוקות יותר היחידות מגיבות לשילובים מורכבים יותר של מאפיינים, ולכן התמונות שממקסמות אותן נראות עשירות ומורכבות יותר. בשכבות ה-fully connected ההשפעה היא גלובלית במיוחד, משום שיחידות אלה משלבות מידע שמקורו בחלקים רבים של התמונה, ולכן האופטימיזציה משנה כמעט את כל התמונה ולא רק אזור קטן במרכזה.

גם בגרף האקטיבציה ניתן לראות הבדלים בין השכבות. בעוד שבשכבות מוקדמות כמו *conv1* האקטיבציה עולה במהירות ומתייצבת בערך נמוך יחסית, בשכבות עמוקות יותר ובשכבות *fc6-fc8* מתקבלות אקטיבציות גבוהות בהרבה. תוצאה זו מחזקת את ההבחנה בין ייצוגים פשוטים ומקומיים בתחילת הרשת לבין ייצוגים מורכבים וגלובליים יותר בשכבות המאוחרות.

מקסום הסתברות של מחלקת פלט מתוך רעש:

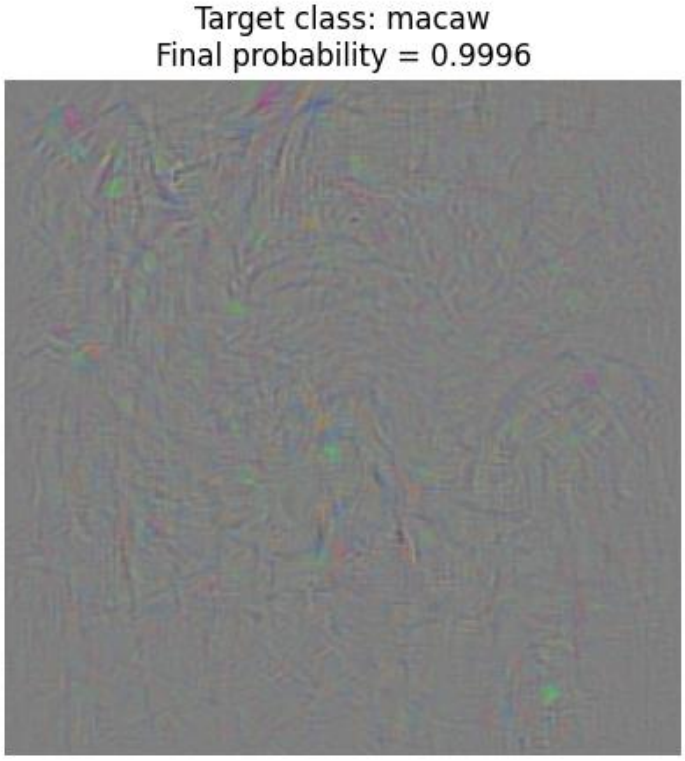
בשלב זה ביצענו Activation Maximization עבור מחלקת פלט ספציפית של AlexNet. בחרנו את המחלקה 'macaw', כלומר תוכי מסוג ארה, והתחלנו מתמונת רעש אקראית, בדומה לסעיפים הקודמים. בשונה מהמקרים הקודמים, שבהם מיקסמנו אקטיבציה של יחידות פנימיות, כאן פונקציית המטרה הייתה הסתברות ה–post SoftMax של מחלקת היעד.

בכל צעד של האופטימיזציה, העברנו את תמונת הקלט דרך AlexNet, חישבנו את הסתברות ה- SoftMax של המחלקה 'macaw', וביצענו gradient ascent על הפיקסלים של התמונה כדי להגדיל הסתברות זו. לאחר כל עדכון חתכנו את ערכי הפיקסלים לטווח התקין ([0,1]), כדי לשמור על תמונה חוקית.

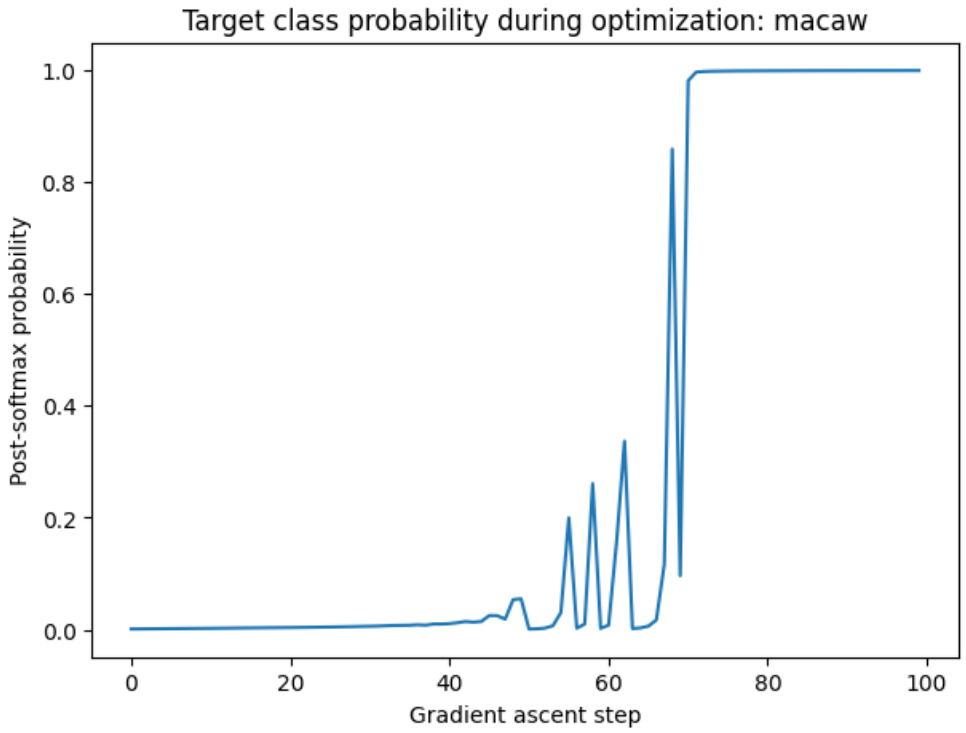
ההסתברות ההתחלתית של המחלקה macaw הייתה 0.00107, ולאחר 100 צעדי אופטימיזציה היא עלתה ל-0.99956. כלומר, הצלחנו להפעיל בצורה חזקה מאוד את המחלקה המבוקשת. גם בבדיקת התחזיות הסופיות macaw, הופיעה כמחלקה בעלת ההסתברות הגבוהה ביותר, עם הסתברות של כ־0.9996.

Initial target probability: 0.00107

Final target probability: 0.99956



תוצאת Activation Maximization עבור מחלקת היעד 'macaw'. לאחר האופטימיזציה, ההסתברות של המחלקה עלתה ל-0.9996.



הסתברות ה-post-SoftMax של המחלקה macaw לאורך תהליך ה-gradient ascent.

בגרף ניתן לראות כי ההסתברות נותרה נמוכה יחסית לאורך חלק גדול מן האופטימיזציה, אך בשלב מאוחר יותר עלתה בחדות והתייצבה קרוב ל-1. תוצאה זו מראה כי גם כאשר מתחילים מתמונת רעש חסרת משמעות חזותית ברורה, ניתן לשנות את הפיקסלים כך שהמודל יזהה את הקלט כמעט בוודאות כמחלקת היעד.

התמונה המתקבלת אינה נראית לבני אדם כמו תוכי ממשי, אלא יותר כמו מרקם חזותי מופשט עם דפוסים עדינים וצבעוניים. הדבר מדגיש שהמודל אינו בהכרח זקוק לייצוג טבעי או אינטואיטיבי מבחינה אנושית, אלא לדפוס פיקסלים שמספיק כדי להפעיל חזק את המחלקה הרצויה ברשת.

נציין כי מאחר שתהליך האופטימיזציה מתחיל מתמונת רעש אקראית, התוצאות עשויות להשתנות בין ריצות שונות. כלומר, בריצות שונות ניתן לקבל דפוסים מעט שונים ואף רמות הצלחה שונות במיקסום ההסתברות של מחלקת היעד.

אתחול מתמונה אמיתית ומקסום מחלקת יעד:

בשלב האחרון חזרנו על רעיון ה-Activation Maximization, אך הפעם לא התחלנו מתמונת רעש אקראית אלא מתמונה אמיתית. בחרנו בתמונת speedboat כתמונת התחלה, והגדרנו כמחלקת יעד את 'Norfolk terrier', כלומר כלב מגזע נורפוק טרייר. מטרת הניסוי הייתה לבדוק האם ניתן לשנות בהדרגה תמונה השייכת לקטגוריה אחת כך שהמודל יסווג אותה כקטגוריה שונה לחלוטין.

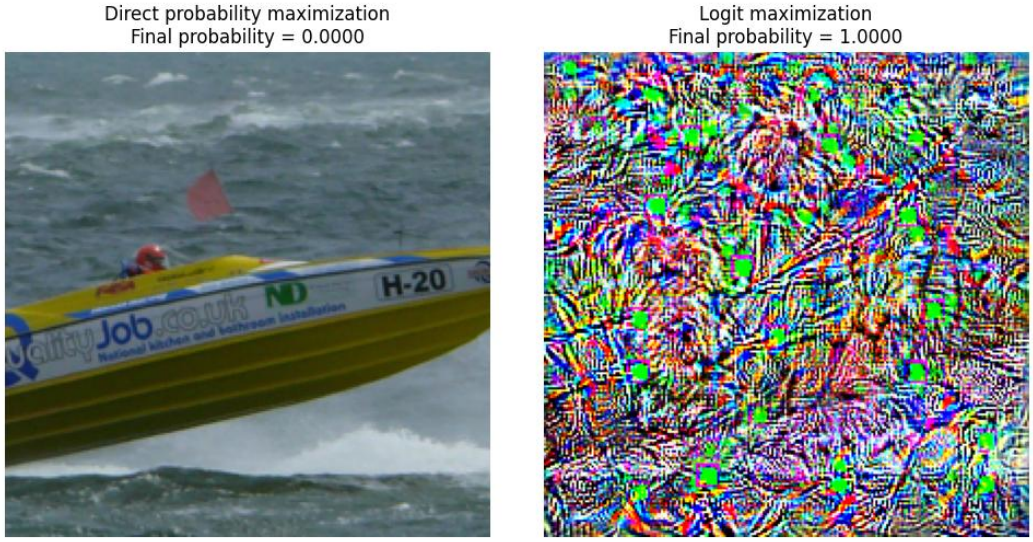


תמונת ההתחלה ששימשה לניסוי: speedboat.

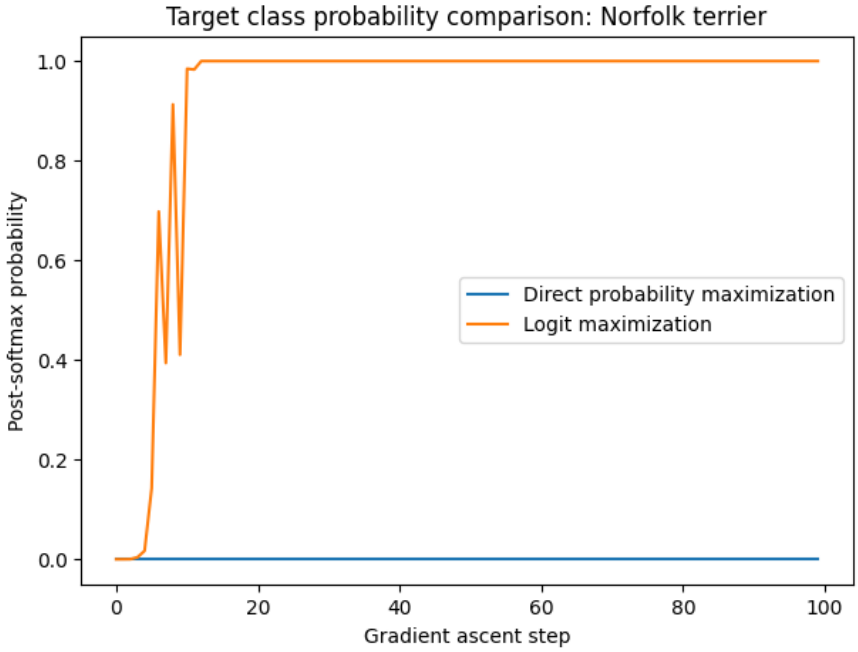
לפני תחילת האופטימיזציה בדקנו את התחזית ההתחלתית של AlexNet עבור התמונה המקורית. כצפוי, המודל סיווג את התמונה כ-speedboat, וההסתברות של מחלקת היעד Norfolk terrier הייתה זניחה מאוד 6.438e-09, כלומר מעשית אפס.

תחילה ניסינו למקסם ישירות את הסתברות ה-post-SoftMax של מחלקת היעד. בניסיון זה ההסתברות של Norfolk terrier נשארה כמעט אפסית לאורך כל האופטימיזציה, והתמונה כמעט לא השתנתה מבחינה משמעותית. תוצאה זו מראה כי מיקסום ישיר של הסתברות ה-SoftMax עלול להיות לא יעיל כאשר מחלקת היעד מתחילה מהסתברות נמוכה מאוד, משום שסיגנל הגרדיאנט עשוי להיות חלש מדי.

לאחר מכן, בהתאם להנחיות המטלה, עברנו למיקסום ה-logit של מחלקת היעד, תוך דיווח על הסתברות ה-post-SoftMax המתקבלת. בניסיון זה התקבלה תוצאה שונה לחלוטין: ההסתברות של Norfolk terrierעלתה מהערך ההתחלתי הזניח עד ל-1.0000, והמחלקה Norfolk terrierהפכה למחלקה המובילה באופן חד-משמעי. גם בגרף ניתן לראות הבדל ברור בין שתי השיטות: בעוד שמיקסום ישיר של ההסתברות נשאר קרוב לאפס, מיקסום ה-logit הוביל לעלייה חדה ומהירה עד להתכנסות לערך קרוב ל-1.



*השוואה בין שתי שיטות אופטימיזציה כאשר מתחילים מתמונת speedboat ומנסים למקסם את המחלקה Norfolk terrier. משמאל: מיקסום ישיר של הסתברות ה-*SoftMax*, שלא הצליח להגדיל את הסתברות היעד. מימין: מיקסום logit, שהוביל להסתברות סופית של 1.0000.



השוואה בין הסתברות מחלקת היעד Norfolk terrier לאורך האופטימיזציה עבור מיקסום ישיר של הסתברות לעומת מיקסום logit.

מבחינה ויזואלית, התמונה שהתקבלה לאחר logit maximization השתנתה באופן קיצוני, ואיבדה את המראה הטבעי של תמונת הסירה המקורית. במקומה התקבל דפוס צפוף, צבעוני ומרקמי מאוד, שנראה לבני אדם כמעט אקראי, אך עבור המודל הוא מהווה קלט שמפעיל בעוצמה את המחלקה Norfolk terrier. תוצאה זו מדגישה פעם נוספת כי הדפוסים שמפעילים רשת נוירונים אינם חייבים להיראות משמעותיים או טבעיים מבחינת תפיסה אנושית.

השוואה זו מראה כי במצבים מסוימים, ובמיוחד כאשר מתחילים מתמונה אמיתית של מחלקה אחרת, מיקסום logit הוא שיטה יעילה ויציבה יותר ממיקסום ישיר של הסתברות ה-*SoftMax*. הסיבה לכך היא שה- logit מספק פונקציית מטרה ישירה יותר ופחות רגישה לדיכוי שנוצר על ידי שאר המחלקות בתוך פעולת ה-SoftMax.